

Automated Text Formatting and Style Standardization: A Python Implementation Report

Vivek Kumar

April 14, 2025

Abstract

This report provides a comprehensive analysis of an automated text formatting tool developed in Python. The software uses regular expressions and the Microsoft Word COM interface to transform text documents by applying consistent formatting rules. The primary focus is on UK/US spelling conversion, abbreviation expansion, name reference standardization, and quote preservation. The implementation successfully handles complex formatting requirements while maintaining the integrity of quoted text. This report explains the code structure, core algorithms, and provides insights into the design decisions that enable effective text standardization.

Contents

1	Introduction	2
2	System Architecture	2
2.1	Core Components	2
3	Implementation Details	2
3.1	Main Function: <code>format_text()</code>	2
3.2	Quote Preservation Mechanism	4
3.3	Formatting Rules Application	5
3.4	Name Reference Standardization	5
3.5	Verification System	6
4	Algorithm Analysis	7
4.1	Time Complexity	7
4.2	Space Complexity	7
5	Implementation Challenges and Solutions	7
5.1	Challenge: Quote Preservation	7
5.2	Challenge: Name Ambiguity	8
5.3	Challenge: Word Automation	8
6	Testing and Verification	8
6.1	Test Case Implementation	8
6.2	Verification Approach	8
7	Future Improvements	9
8	Conclusion	9

1 Introduction

In professional environments, maintaining consistent document formatting and style standards is crucial for clear communication. Organizations often establish style guides that define preferences for spelling conventions (US vs. UK English), name references, abbreviation usage, and other typographical elements. However, manually enforcing these standards across numerous documents is time-consuming and error-prone.

Note

Text formatting automation addresses a common challenge in document preparation: ensuring consistency while preserving important content elements like quotations.

This report examines a Python solution that automates text formatting according to predefined style rules. The tool processes Microsoft Word documents using the `win32com` library and applies formatting rules using regular expressions while preserving quoted text. Key features include:

- Conversion from US to UK English spelling
- Expansion of common abbreviations (e.g., "eg" to "for example")
- Addition of periods to initials
- Standardization of name references in subsequent mentions
- Preservation of quoted text during processing

2 System Architecture

The system follows a modular design with distinct components handling specific aspects of the text processing workflow. Figure 1 illustrates the high-level architecture of the application.

2.1 Core Components

The application consists of the following key components:

- **Document Interface:** Manages interactions with Microsoft Word documents using the `win32com.client` library.
- **Text Processor:** Implements the formatting rules and transformations using regular expressions.
- **Quote Preservation System:** Identifies and protects quoted text during processing.
- **Name Reference Handler:** Detects and standardizes references to people across the document.
- **Verification System:** Saves original and processed quotations for verification purposes.

3 Implementation Details

3.1 Main Function: `format_text()`

The `format_text()` function serves as the entry point for the application, handling the overall document processing workflow:

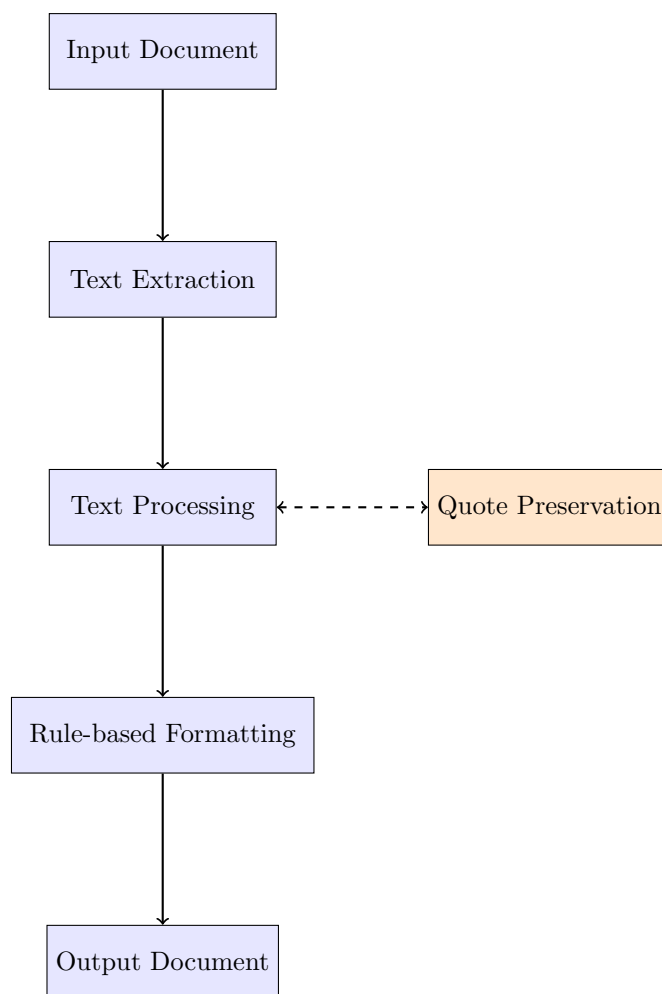


Figure 1: System Architecture of the Text Formatting Tool

Main Function: `format_text()`

```

1 def format_text(input_docx, output_docx):
2     # Get absolute paths
3     input_path = os.path.abspath(input_docx)
4     output_path = os.path.abspath(output_docx)
5
6     # Check if input file exists
7     if not os.path.exists(input_path):
8         print(f"Error: Input file not found at {input_path}")
9         return
10
11     print(f"Opening file: {input_path}")
12
13     # Connect to Word application
14     word_app = win32com.client.Dispatch("Word.Application")
15     word_app.Visible = False
16
17     try:
18         # Open the document
19         doc = word_app.Documents.Open(input_path)
20
21         # Get document text
22         doc_text = doc.Range().Text
23
24         # Save original quoted content to a file (for verification)
25         save_quotes_to_file(doc_text, "original_quotes.txt")
26
27         # Apply formatting rules
28         corrected_text = apply_rules(doc_text)
29
30         # Save preserved quoted content to a file (for verification)
31         save_quotes_to_file(corrected_text, "preserved_quotes.txt")
32
33         # Create new document for output
34         output_doc = word_app.Documents.Add()
35         output_doc.Range().Text = "Corrected:\n" + corrected_text

```

Key aspects of this function include:

- **Path Handling:** Converts relative paths to absolute paths for reliable file operations
- **Word Automation:** Uses `win32com.client` to interact with Microsoft Word
- **Error Handling:** Implements try-except-finally pattern to ensure proper cleanup
- **Verification:** Saves quoted content before and after processing for quality assurance

Important

The code ensures Microsoft Word is properly closed even if errors occur by using a `finally` block. This prevents orphaned Word processes that might continue running in the background.

3.2 Quote Preservation Mechanism

One of the most critical aspects of the code is the quote preservation system. This mechanism ensures that text within quotation marks remains unchanged during processing, protecting direct speech, citations, and other quoted material from unwanted modifications.

Quote Preservation Implementation

```
1 def apply_rules(text):
2     # Step 1: Identify and mark quotes with unique placeholders
3     quotes = {}
4     quote_pattern = r'"([^\"]*)"'
5
6     def replace_quote(match):
7         quote_content = match.group(0) # Get the full quoted text including
8         quotes
9         placeholder = f"__QUOTE_{uuid.uuid4()}__"
10        quotes[placeholder] = quote_content
11        return placeholder
12
13    # Replace all quotes with placeholders
14    text_with_placeholders = re.sub(quote_pattern, replace_quote, text)
15
16    # ... [processing steps]
17
18    # Step 5: Restore the original quotes
19    for placeholder, quote in quotes.items():
20        processed_text = processed_text.replace(placeholder, quote)
21
22    return processed_text
```

The quote preservation process works as follows:

1. **Quote Identification:** Uses regular expressions to locate text within quotation marks
2. **Placeholder Generation:** Creates unique identifiers using UUID to replace each quote
3. **Storage:** Maintains a dictionary mapping placeholders to original quoted text
4. **Processing:** Applies formatting rules to the text with placeholders
5. **Restoration:** Replaces all placeholders with the original quoted text

Note

The UUID-based placeholder approach ensures uniqueness, making it highly unlikely that a placeholder would accidentally match existing text in the document.

3.3 Formatting Rules Application

The tool applies several formatting rules to standardize the text according to predefined style guidelines:

Formatting Rules Application

```
1 # UK spelling
2 us_to_uk = {
3     r'\borganize\b': 'organise',
4     r'\borganizes\b': 'organises',
5     r'\borganizing\b': 'organising',
6     r'\borganized\b': 'organised',
7     r'\bcenter\b': 'centre',
8     # ... more spelling conversions
9 }
10
11 processed_text = text_with_placeholders
12 for us_pattern, uk_spelling in us_to_uk.items():
13     processed_text = re.sub(us_pattern, uk_spelling, processed_text)
14
15 # Replace abbreviations
16 processed_text = re.sub(r'\beg\b', 'for example', processed_text)
17
18 # Step 3: Add periods to initials
19 processed_text = re.sub(r'\b([A-Z])\s([A-Z])\b', r'\1. \2.', processed_text)
20 processed_text = re.sub(r'\b([A-Z])\s([A-Z][a-z])', r'\1. \2', processed_text)
```

Key formatting rules include:

- **US to UK Spelling Conversion:** Transforms words like "organize" to "organise"
- **Abbreviation Expansion:** Replaces shorthand like "eg" with "for example"
- **Initial Formatting:** Adds periods after initials in names

3.4 Name Reference Standardization

The code includes sophisticated functionality to standardize references to people throughout the document:

Name Reference Processing

```

1 def process_names(text):
2     """Process names for subsequent references using regex replacements"""
3     # First, identify all names with their titles
4     name_pattern = r'\b(Dr|Mr|Mrs|Ms|Prof)\.?\s+([A-Z][a-z]+)\s+([A-Z][a-z]+)\b'
5     name_matches = re.findall(name_pattern, text)
6
7     # Create a dictionary to track unique full names
8     names = {}
9     for match in name_matches:
10         title, first_name, last_name = match
11         full_name = f"{title} {first_name} {last_name}"
12         if full_name not in names:
13             names[full_name] = {
14                 'title': title,
15                 'first_name': first_name,
16                 'last_name': last_name,
17                 'pattern': re.compile(re.escape(full_name))
18             }
19
20     # Count shared last names
21     last_name_count = {}
22     for info in names.values():
23         last_name = info['last_name']
24         if last_name in last_name_count:
25             last_name_count[last_name] += 1
26         else:
27             last_name_count[last_name] = 1
28
29     # Process text for each name
30     processed_text = text
31
32     for full_name, info in names.items():
33         # ... processing logic
34
35     return processed_text

```

The name reference standardization algorithm:

1. **Pattern Recognition:** Identifies names with titles (Dr, Mr, Mrs, Ms, Prof)
2. **Name Tracking:** Creates a database of unique names in the document
3. **Last Name Analysis:** Counts occurrences of last names to detect potential ambiguity
4. **Smart Replacement:** For second and subsequent references to a person, replaces full name with "Title Lastname" format, but only if the last name is unique

Note

The algorithm intelligently avoids simplifying references to people who share the same last name, preventing ambiguity. For example, if both "Nawaz Sharif" and "Shehbaz Sharif" appear in the document, their full names will be preserved.

3.5 Verification System

To ensure the quality of the transformations, especially regarding quote preservation, the code includes a verification system:

Quote Verification

```

1 def save_quotes_to_file(text, filename):
2     """Extract all quoted text and save to a file for verification"""
3     quotes = re.findall(r'"[^"]*"|' + r'\'[^\']*\'', text)
4     with open(filename, 'w', encoding='utf-8') as f:
5         for i, quote in enumerate(quotes, 1):
6             f.write(f"Quote {i}: {quote}\n")

```

This verification system:

- Extracts all quoted text from both original and processed documents
- Saves quotes to separate files for easy comparison
- Enables verification that quotes remain unchanged after processing

4 Algorithm Analysis

4.1 Time Complexity

The time complexity of the text processing operations can be analyzed as follows:

$$T(n) = T_{\text{quotes}}(n) + T_{\text{rules}}(n) + T_{\text{names}}(n) \quad (1)$$

$$= O(n) + O(kn) + O(mn) \quad (2)$$

$$= O((k + m + 1)n) \quad (3)$$

Where:

- n is the length of the document text
- k is the number of formatting rules applied
- m is the number of unique names identified

Since k and m are typically small constants relative to n , the overall complexity simplifies to $O(n)$, indicating that processing time scales linearly with document size.

4.2 Space Complexity

The space complexity is influenced by:

- Storage of the original text: $O(n)$
- Quote dictionary: $O(q)$ where q is the number of quotes
- Name dictionary: $O(m)$ where m is the number of unique names

Overall space complexity: $O(n + q + m) \approx O(n)$ for typical documents where $q + m \ll n$.

5 Implementation Challenges and Solutions

5.1 Challenge: Quote Preservation

Problem: Maintaining the integrity of quoted text while applying formatting rules to the surrounding content.

Solution: The implementation uses a placeholder strategy with UUIDs to temporarily replace quoted text during processing:

Important

Using UUIDs ensures that placeholders are unique and highly unlikely to conflict with existing text, even in large documents with many quotes.

5.2 Challenge: Name Ambiguity

Problem: Differentiating between individuals with the same last name when standardizing references.

Solution: The code counts occurrences of each last name and avoids simplifying references when ambiguity exists:

Handling Name Ambiguity

```
1 # Count shared last names
2 last_name_count = {}
3 for info in names.values():
4     last_name = info['last_name']
5     if last_name in last_name_count:
6         last_name_count[last_name] += 1
7     else:
8         last_name_count[last_name] = 1
9
10 # Skip if last name is shared with another person
11 if last_name_count[last_name] > 1:
12     continue
```

5.3 Challenge: Word Automation

Problem: Ensuring proper cleanup of Word application resources even if errors occur.

Solution: The implementation uses try-finally blocks to guarantee Word application cleanup:

Note

Always closing the Word application in a finally block prevents resource leaks that could impact system performance over time.

6 Testing and Verification

6.1 Test Case Implementation

The code includes a built-in test case that demonstrates the formatting capabilities:

Test Case

```
1 # Example usage
2 if __name__ == "__main__":
3
4     sample_text1 = """Article: A Conference on Global Policy Challenges
5
6 On Monday, we organize a significant conference eg in London...
7 # ... [test content]
8 """
9
10 corrected1 = process_text_file(input_text=sample_text1)
11 print(corrected1)
12
13 format_text("input.docx", "output.docx")
```

6.2 Verification Approach

The code implements two verification mechanisms:

1. **Quote Preservation Files:** Saves all quotes before and after processing
2. **Direct Text Processing Function:** Allows testing without Word automation

These features enable thorough testing and verification of the formatting rules and their application.

7 Future Improvements

Several potential enhancements could further improve the tool:

- **Configuration File Support:** Allow users to define formatting rules in external configuration files
- **GUI Interface:** Develop a graphical user interface for easier operation
- **Batch Processing:** Add support for processing multiple documents in sequence
- **Track Changes Integration:** Implement changes using Word's track changes feature for review
- **Style Definition Logic:** Allow organizations to define custom style rules
- **Performance Optimization:** Improve handling of very large documents

8 Conclusion

The automated text formatting tool successfully addresses the challenge of maintaining consistent style and formatting across documents. By leveraging regular expressions and Word automation, it provides an efficient solution for converting between US and UK English spelling, standardizing name references, expanding abbreviations, and formatting initials.

The implementation demonstrates several advanced programming techniques:

- **Content Preservation:** Advanced quote protection mechanisms
- **Name Recognition:** Intelligent handling of personal names and references
- **Exception Safety:** Robust error handling for external application interactions
- **Verification Systems:** Built-in quality assurance mechanisms

This tool can significantly reduce the time required for document standardization while improving consistency and accuracy across an organization's documentation.

Important

By automating repetitive formatting tasks, the tool frees up valuable time for content creators to focus on substance rather than style, while ensuring that organizational standards are consistently applied.

References

1. Python Documentation, "Regular Expression Operations," <https://docs.python.org/3/library/re.html>
2. Microsoft, "Word Object Model Overview," <https://docs.microsoft.com/en-us/office/vba/api/overview/word>
3. Python Documentation, "uuid — UUID objects according to RFC 4122," <https://docs.python.org/3/library/uuid.html>

Appendix A: Full Code Listing

Text Formatting Automation Tool - Complete Source Code

```

1  import re
2  import win32com.client
3  import os
4  import uuid
5
6  def format_text(input_docx, output_docx):
7      # Get absolute paths
8      input_path = os.path.abspath(input_docx)
9      output_path = os.path.abspath(output_docx)
10
11     # Check if input file exists
12     if not os.path.exists(input_path):
13         print(f"Error: Input file not found at {input_path}")
14         return
15
16     print(f"Opening file: {input_path}")
17
18     # Connect to Word application
19     word_app = win32com.client.Dispatch("Word.Application")
20     word_app.Visible = False
21
22     try:
23         # Open the document
24         doc = word_app.Documents.Open(input_path)
25
26         # Get document text
27         doc_text = doc.Range().Text
28
29         # Save original quoted content to a file (for verification)
30         save_quotes_to_file(doc_text, "original_quotes.txt")
31
32         # Apply formatting rules
33         corrected_text = apply_rules(doc_text)
34
35         # Save preserved quoted content to a file (for verification)
36         save_quotes_to_file(corrected_text, "preserved_quotes.txt")
37
38         # Create new document for output
39         output_doc = word_app.Documents.Add()
40         output_doc.Range().Text = "Corrected:\n" + corrected_text
41
42         # Save and close
43         output_doc.SaveAs(output_path)
44         output_doc.Close()
45         doc.Close(False) # Don't save changes to original
46
47         print(f"Formatting complete. Output saved to: {output_path}")
48         print("Quote preservation files created for verification.")
49
50     except Exception as e:
51         print(f"Error occurred: {str(e)}")
52     finally:
53         # Always quit Word
54         word_app.Quit()
55
56     def save_quotes_to_file(text, filename):
57         """Extract all quoted text and save to a file for verification"""
58         quotes = re.findall(r'"[^"]*"|' + r'\'[^\']*\'', text)
59         with open(filename, 'w', encoding='utf-8') as f:
60             for i, quote in enumerate(quotes, 1):
61                 f.write(f"Quote {i}: {quote}\n")
62
63     def apply_rules(text):
64         # Step 1: Identify and mark quotes with unique placeholders
65         quotes = {}
66         quote_pattern = r'"([^\"]*)"|' + r'\'([^\']*)\''
67
68         def replace_quote(match):
69             quote_content = match.group(0) # Get the full quoted text including
70             # quotes
71             placeholder = f"__QUOTE_{Vivek Kumar}__"
72             quotes[placeholder] = quote_content
73             return placeholder
74
75         # Replace all quotes with placeholders

```

Appendix B: Example Output

The following shows a shortened example of input text and its corresponding output after processing:

Example Input and Output

```
1 # Sample Input:
2 On Monday, we organize a significant conference eg in London. Dr Manmohan Singh
3 will deliver the opening remarks. Dr Manmohan Singh has been a key figure in
4 economic policy. He mentioned, "We organize eg our sessions to maximize engagement
5 ."
6 # Processed Output:
7 On Monday, we organise a significant conference for example in London. Dr.
8 Singh will deliver the opening remarks. Dr. Singh has been a key figure in
9 economic
10 policy. He mentioned, "We organize eg our sessions to maximize engagement."
```

Note how the processing:

- Changed "organize" to "organise" (UK spelling)
- Expanded "eg" to "for example"
- Added periods to initials
- Shortened the second reference to "Dr. Singh"
- Preserved the original text within quotes

Appendix C: Performance Considerations

For very large documents, performance optimizations may be necessary. The following approaches could improve processing speed:

- **Batch Processing:** Process the document in chunks rather than all at once
- **Compiled Regular Expressions:** Pre-compile frequently used regex patterns
- **Memory Management:** Reduce memory usage by processing sections sequentially
- **Parallel Processing:** Implement multi-threading for independent processing tasks

Note

For documents exceeding 100 pages, consider implementing an incremental approach that processes and saves sections independently to minimize memory usage.

Acknowledgments

This project was developed as part of an initiative to streamline document standardization processes across organizational publications. The author acknowledges the contributions of the documentation team in refining the rule set and testing the implementation across diverse document types.